PUC-RIO

Fabiano Carvalho Costa Júnior

Relatório de Software Básico (INF1018)
Chamadas de Funções em Assembly x86-64

Rio de Janeiro
2026

1 Invocação da Função `system`

O primeiro exercício traduziu a função `main` para Assembly para executar o comando `system("ls -ls")`. O ponteiro da string foi passado no registrador `%rdi`, conforme a convenção System V AMD64.

1.1 Código Assembly (`ex1.s`)

```
.data
s1: .string "ls -ls"

.text
.globl main
main:
  /* prologo */
    pushq %rbp
    movq  %rsp, %rbp

    leaq  s1(%rip), %rdi
    movl  $0, %eax
    call  system@PLT

    movl  $0, %eax

  /* finalizacao */
    leave
    ret
```

Compilação e Execução:

```
gcc -Wall -no-pie -o ex1 ex1.s
./ex1
```

—

2 Iteração sobre Vetores e Preservação de Registradores

O segundo exercício integrou o módulo `filtro.c` com uma `main` em Assembly. Os registradores *callee-saved* `%rbx` (ponteiro `p`) e `%r12d` (contador `i`) foram utilizados para preservar seus valores durante as chamadas às funções `filtro` e `printf`.

2.1 Código Assembly (`mainasm.s`)

```
.data
nums: .int 3, -5, 7, 8, -2
s1:   .string "%d\n"

.text
.globl main
main:
  /* prologo */
     pushq %rbp
     movq  %rsp, %rbp
     subq  $16, %rsp
     movq  %rbx, -8(%rbp)
     movq  %r12, -16(%rbp)

     leaq  nums(%rip), %rbx     # p = nums
     movl  $0, %r12d            # i = 0

.L_loop:
```

```
    cmpl  $5, %r12d
    jge   .L_fim

    # filtro(*p, LIM)
    movl  (%rbx), %edi
    movl  $1, %esi
    call  filtro@PLT

    # printf("%d\n", resultado)
    movl  %eax, %esi
    leaq  s1(%rip), %rdi
    movl  $0, %eax
    call  printf@PLT

    addq  $4, %rbx            # p++
    incl  %r12d               # i++
    jmp   .L_loop

.L_fim:
    movl  $0, %eax

  /* finalizacao */
    movq  -8(%rbp), %rbx
    movq  -16(%rbp), %r12
    leave
    ret
```

2.2 Resultado

A versão em Assembly (`gcc -Wall -no-pie -o ex2asm filtro.c mainasm.s`) gerou a mesma saída do código original em C, zerando os elementos menores que 1:

```
3
0
7
8
0
```

3 Conclusão

O laboratório consolidou a passagem de argumentos via registradores (`%rdi`, `%rsi`), o uso de registradores *callee-saved* para manter variáveis em laços com chamadas de função e a manipulação adequada do quadro de pilha (*stack frame*).